B.M.S COLLEGE OF ENGINEERING

(An Autonomous Institution Affiliated to VTU, Belagavi) **Post Box**

No.: 1908, Bull Temple Road, Bengaluru – 560 019

DEPARTMENT OF MACHINE LEARNING

Academic Year: 2025-2026 (Session: Aug 2025- Dec 2026)

Deep Learning
(24AM5PCDEL)

ALTERNATIVE ASSESSMENT TOOL (AAT)

"Car vs Bike Image Classification using MobileNetV2"

Submitted by

Student Name: Team Member 1: Team Member 2:	 Nithin E T -
USN: Team Member 1: Team Member 2:	 1BM23AI114 --
Semester & Section:	5th Sem 5B section
Student Signature: Team Member 1: Team Member 2:	

Valuation Report **(to be filled by the faculty)**

Faculty In-charge:	**Prof. Sudhindra R**		
Faculty Signature with date:		Score:	

Table of Contents

1 INTRODUCTION

- **I. Detail of the Deep learning Architecture choosen(with architecture diagram)**
- **II. Highlight the key features of the Architecture**
- **III. Detail of Application built on Deep learning Architecture(Code/Low code/No code)**

Note: If simulation tool used then also describe the architecture with application

2 SCREENSHOTS OF OUTPUT

1. INTRODUCTION

Image classification is one of the most fundamental tasks in computer vision. It aims to automatically assign a semantic label to an input image. In this project, we address a binary classification problem: distinguishing between car images and bike images.

Such a model has applications in:

- Intelligent traffic monitoring systems
- Automated parking management
- Vehicle statistics and analytics
- Smart surveillance and transportation systems

The project uses a Kaggle Car vs Bike classification dataset, containing images of vehicles from two categories: Car and Bike. These images vary in viewpoint, background and illumination, which makes the task non-trivial and a good test case for deep learning models.

To solve this problem, we design a model that combines:

- A pre-trained MobileNetV2 CNN as a feature extractor, and
- A custom Residual MLP (ResMLP) classification head implemented from scratch.

This approach leverages transfer learning for robust visual features and uses a flexible MLP structure with residual connections for final classification.

I. DETAILS OF THE DEEP LEARNING ARCHITECTURE CHOSEN

The complete pipeline followed in the project can be summarized as:

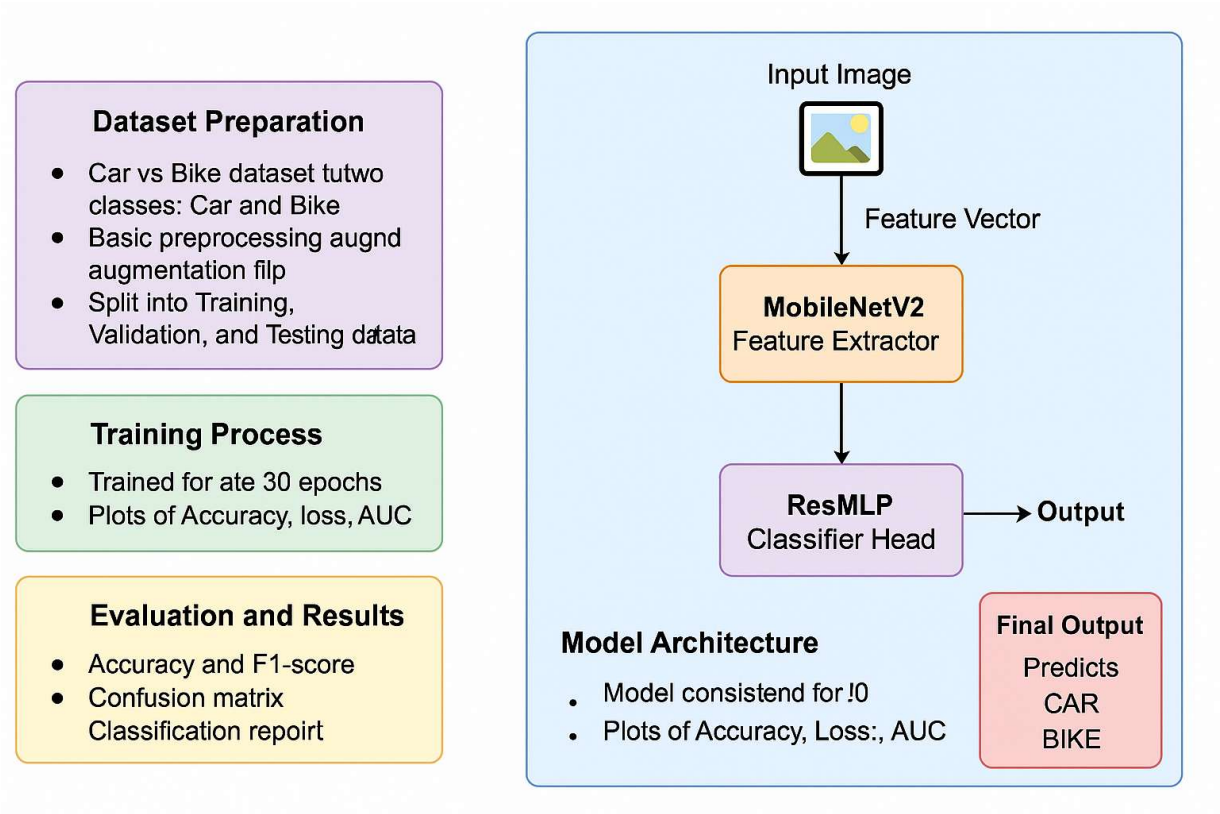
1. **Dataset Preparation**
    - Read all images from Car-Bike-Dataset/.
    - Filter valid image files (jpg, jpeg, png, tiff).
    - Build a DataFrame with two columns: filepaths and labels.
    - Split into train (80%), validation (10%), and test (10%) sets using train_test_split with stratification.
2. **Image Generators**
    - Use ImageDataGenerator to create train_ds, valid_ds, and test_ds.
    - Apply horizontal flip augmentation on training data.

- Resize images to a fixed target size for training.

3. **Feature Extraction using MobileNetV2**
   - Load tf.keras.applications.MobileNetV2 with:
     - include_top=False
     - Pre-trained on ImageNet (weights='imagenet')
     - pooling='max' (global max pooling)
   - Freeze the base model (base_model.trainable = False).
4. **Custom ResMLP Head**
   - Flattened feature vectors from MobileNetV2 are passed to a custom ResMLP module.
   - ResMLP consists of:
     - Initial dense layer with ReLU activation and batch normalization.
     - A stack of ResMLPBlock layers organized according to block_list=[2, 2, 2].
     - Final dense layer with softmax activation to output probabilities for 2 classes: Car and Bike.
5. **Model Training and Evaluation**
   - Compile model with:
     - Optimizer: Adam
     - Loss: CategoricalCrossentropy(from_logits=False)
     - Metrics: accuracy, AUC
   - Train the model for 30 epochs with validation data.
   - Plot loss, accuracy and AUC curves.
   - Evaluate on test set using:
     - Overall test accuracy

- F1-score
- Confusion matrix
- Classification report (precision, recall, f1-score per class)

2. Architecture Diagram



II. KEY FEATURES OF THE ARCHITECTURE

□ Transfer Learning with MobileNetV2

- Uses a light-weight CNN pre-trained on ImageNet.

- Enables the model to exploit rich, general-purpose visual features without training a large CNN from scratch.
- Reduces training time and requirement of very large datasets.

Custom Residual MLP (ResMLP) Head

- Designed using **Dense layers**, **Batch Normalization**, and **residual connections**.
- Residual paths help enrich feature transformation while keeping training stable.
- The custom **GELU** activation function is applied in the residual path to improve non-linearity.

Stratified Data Splitting

- Ensures both Car and Bike classes are proportionally represented in train, validation, and test splits.

ImageDataGenerator-based Pipeline

- On-the-fly augmentation such as horizontal flip provides better generalization.
- Generators load batches directly from disk, which is memory efficient.

Comprehensive Evaluation Metrics

- Apart from accuracy, the model computes:
    - AUC (Area Under ROC Curve) during training
    - Confusion matrix
    - Classification report (precision, recall, F1-score)
    - Weighted F1-score for overall performance.

Modular Code Design

- Separate functions for:
    - Dataframe creation (make_dataframes)
    - Generator creation (make_gens)
    - Custom layers (ResMLPBlock, ResMLP)
    - Model creation (make_model)
    - Training curve plotting (plot_auc_acc_loss)
    - Testing and evaluation (predictor)

III. Detail of Application Built Using GAN (Image-to-Image Translation):

The application developed in this project performs Car vs Bike image classification using a combination of MobileNetV2 for feature extraction and a custom Residual MLP classifier for final prediction.

4.1 Dataset Preparation

The Car vs Bike dataset contains two classes: Car and Bike.

- All images were cleaned, validated, and organized into a structured table.
- The dataset was split into Training (80%), Validation (10%), and Testing (10%) sets.
- Basic image preprocessing and simple augmentation (horizontal flip) were applied to improve generalization.

4.2 Model Architecture

The model consists of two main components:

1. MobileNetV2 Feature Extractor

- A pre-trained deep CNN used only to extract image features.
- Its layers remain frozen, making training faster and preventing overfitting.
- Converts images into meaningful feature vectors**.**

2. ResMLP Classification Head

- A lightweight series of fully connected layers with residual connections.
- Learns to differentiate between Car and Bike features.
- Ends with a softmax layer that outputs the class probabilities.

4.3 Training Process (Short Summary)

- The model was trained for 30 epochs using the training dataset.
- Validation accuracy and loss were monitored to avoid overfitting.

- Performance curves (accuracy, loss, AUC) were plotted for analysis.

4.4 Evaluation and Results

- **The model was tested on unseen test images.**
- **Evaluation metrics included:**
    - **Accuracy**
    - **F1-score**
    - **Confusion matrix**
    - **Classification report**

The model achieved high accuracy in distinguishing Car vs Bike, showing strong classification performance.

4.5 Final Output

The final trained model can take any new vehicle image and correctly predict whether it is a:

- CAR, or
- BIKE

This makes it useful for transportation analytics, automated monitoring, and vehicle classification tasks.

Code Implementation :

```python
import numpy as np
import pandas as pd
import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
        break
```

Python

```python
import tensorflow as tf
from tqdm import tqdm
import pandas as pd
import numpy as np
import os, shutil, math, cv2, json, random, re, sys, datetime
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from PIL import Image
import glob
from keras.layers import *
from tensorflow.keras import Model, layers, initializers
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, f1_score, classification_report
from sklearn.metrics import confusion_matrix
import seaborn as sns
from keras.preprocessing.image import ImageDataGenerator
```

Python

```python
sdir = '/kaggle/input/car-vs-bike-classification-dataset/Car-Bike-Dataset/'

def make_dataframes(sdir):
    bad_images = []
    good_ext = ['jpg', 'jpeg', 'png', 'tiff']
    filepaths = []
    labels = []
    classes = sorted(os.listdir(sdir))
    for klass in classes:
        classpath = os.path.join(sdir, klass)
        flist = sorted(os.listdir(classpath))
        desc = f'{klass:25}'
        for f in tqdm(flist, ncols=110, desc=desc, unit='file', colour='blue'):
            fpath = os.path.join(classpath, f)
            f1 = f.lower()
            index = f1.rfind('.')
            ext = f1[index + 1:]
            if ext in good_ext:
                try:
                    img = cv2.imread(fpath)
                    shape = img.shape
                    filepaths.append(fpath)
                    labels.append(klass)
                except:
                    bad_images.append(fpath)
                    print(f'defective image file: ', fpath)
            else:
                bad_images.append(fpath)
    Fseries = pd.Series(filepaths, name='filepaths')
    Lseries = pd.Series(labels, name='labels')
    df = pd.concat([Fseries, Lseries], axis=1)

    train_df, dummy_df = train_test_split(df, train_size=.8, shuffle=True, random_state=123, stratify=df['labels'])
    valid_df, test_df = train_test_split(dummy_df, train_size=.5, shuffle=True, random_state=123,
                                         stratify=dummy_df['labels'])
    classes = sorted(train_df['labels'].unique())
    class_count = len(classes)
    sample_df = train_df.sample(n=50, replace=False)

    ht = 0
    wt = 0
    count = 0
    for i in range(len(sample_df)):
        fpath = sample_df['filepaths'].iloc[i]
        try:
            img = cv2.imread(fpath)
            h = img.shape[0]
            w = img.shape[1]
            wt += w
            ht += h
            count += 1
        except:
            pass
    have = int(ht / count)
    wave = int(wt / count)
    aspect_ratio = have / wave
    print('number of classes in processed dataset= ', class_count)
    counts = list(train_df['labels'].value_counts())
    print(' the maximum files in any class in train_df is ', max(counts),
          ' the minimum files in any class in train_df is ', min(counts))
    print('train_df length: ', len(train_df), ' test_df length: ', len(test_df), ' valid_df length: ', len(valid_df))
    print('average image height= ', have, ' average image width= ', wave, ' aspect ratio h/w= ', aspect_ratio)
    return train_df, test_df, valid_df, classes, class_count

train_df, test_df, valid_df, classes, class_count = make_dataframes(sdir=sdir)
```

Python

```python
batch_size = 16
img_size = (224, 300)

def make_gens(batch_size, train_df, test_df, valid_df, img_size):
    '''
    trgen = ImageDataGenerator(horizontal_flip=True)
    t_and_v_gen = ImageDataGenerator()
    msg = '{0:70s} for train generator'.format(' ')
    print(msg, '\r', end='')
    train_ds = trgen.flow_from_dataframe(train_df, x_col='filepaths', y_col='labels',
                                         target_size=img_size, class_mode='categorical',
                                         color_mode='rgb', batch_size=batch_size, shuffle=True)

    msg = '{0:70s} for valid generator'.format(' ')
    print(msg, '\r', end='')
    valid_ds = t_and_v_gen.flow_from_dataframe(valid_df, x_col='filepaths', y_col='labels',
                                         target_size=img_size, class_mode='categorical',
                                         color_mode='rgb', batch_size=batch_size, shuffle=False)

    test_len = len(test_df)
    test_batch_size = sorted([int(test_len / n) for n in range(1, test_len + 1)
                              if test_len % n == 0 and test_len / n<=80], reverse=True)[0]
    test_steps = int(test_len / test_batch_size)
    msg = '{0:70s} for test generator'.format(' ')
    print(msg, '\r', end='')
    test_ds = t_and_v_gen.flow_from_dataframe(test_df, x_col='filepaths', y_col='labels',
                                         target_size=img_size, class_mode='categorical',
                                         color_mode='rgb', batch_size=batch_size, shuffle=False)

    classes = list(train_ds.class_indices.keys())
    class_count = len(classes)

    print('test batch size: ', test_batch_size, 'test steps: ', test_steps, 'number of classes : ', class_count)


    return train_ds, test_ds, valid_ds

train_ds, test_ds, valid_ds = make_gens(batch_size, train_df, test_df, valid_df, img_size)
```

```python
def GELU(x):
    res = 0.5 * x * (1 + tf.nn.tanh(math.sqrt(2 / math.pi) * (x + 0.044715 * (x ** 3))))
    return res


class ResMLPBlock(tf.keras.layers.Layer):


    def __init__(self, units, residual_path):
        super(ResMLPBlock, self).__init__()
        self.residual_path = residual_path
        self.D1 = Dense(units, activation='relu')
        self.D2 = Dense(units, activation='relu')

        if self.residual_path:
            self.D3 = Dense(units)
            self.D4 = Dense(units)

    def call(self, inputs):
        residual = inputs

        x = self.D1(inputs)
        y = self.D2(x)

        if self.residual_path:
            residual = self.D3(inputs)
            residual = GELU(residual)
            residual = self.D4(residual)
            residual = GELU(residual)

        output = y + residual
        return output


class ResMLP(tf.keras.layers.Layer):
    def __init__(self, initial_filters, block_list, num_classes):
        super(ResMLP, self).__init__()
        self.initial_filters = initial_filters
        self.block_list = block_list

        self.D1 = Dense(self.initial_filters, activation='relu')
        self.B1 = BatchNormalization()

        self.blocks = tf.keras.models.Sequential()
        for block_id in range(len(block_list)):
            for layer_id in range(block_list[block_id]):
                if block_id != 0 and layer_id == 0:
                    block = ResMLPBlock(units=self.initial_filters, residual_path=True)
                else:
                    block = ResMLPBlock(units=self.initial_filters, residual_path=False)
                self.blocks.add(block)
            self.initial_filters *= 2

        self.D2 = Dense(num_classes, activation='softmax')


    def call(self, inputs, training=None):
        x = self.D1(inputs)
        x = self.B1(x, training=training)
        x = self.blocks(x)
        y = self.D2(x)
        return y
```

```python
def make_model():
    base_model = tf.keras.applications.MobileNetV2(include_top=False, weights='imagenet', pooling='max', input_shape=(400, 400, 3))
    base_model.trainable=False
    x = base_model.output
    output = ResMLP(initial_filters=32, block_list=[2, 2, 2], num_classes=2)(x)
    return tf.keras.models.Model(inputs=base_model.input, outputs=output)

net = make_model()
net.compile(optimizer='adam', loss=tf.keras.losses.CategoricalCrossentropy(from_logits=False), metrics=['accuracy', 'AUC'])

history = net.fit(train_ds, batch_size=16, epochs=30, validation_data=valid_ds)
```

```python
def plot_auc_acc_loss(history, epochs):
    tacc = history.history['accuracy']
    tloss = history.history['loss']
    tauc = history.history['auc']

    vacc = history.history['val_accuracy']
    vloss = history.history['val_loss']
    vauc = history.history['val_auc']

    Epochs = [i for i in range(epochs)]

    plt.style.use('fivethirtyeight')
    fig, axes = plt.subplots(nrows=1, ncols=3, figsize=(24, 8))
    axes[0].plot(Epochs, tloss, 'r', label='Training loss')
    axes[0].plot(Epochs, vloss, 'g', label='Validation loss')
    axes[0].set_title('Training and Validation Loss')
    axes[0].set_xlabel('Epochs', fontsize=18)
    axes[0].set_ylabel('Loss', fontsize=18)
    axes[0].legend()

    axes[1].plot(Epochs, tacc, 'r', label='Training Accuracy')
    axes[1].plot(Epochs, vacc, 'g', label='Validation Accuracy')
    axes[1].set_title('Training and Validation Accuracy')
    axes[1].set_xlabel('Epochs', fontsize=18)
    axes[1].set_ylabel('Accuracy', fontsize=18)
    axes[1].legend()

    axes[2].plot(Epochs, tauc, 'r', label='Training AUC')
    axes[2].plot(Epochs, vauc, 'g', label='Validation AUC')
    axes[2].set_title('Training and Validation AUC')
    axes[2].set_xlabel('Epochs', fontsize=18)
    axes[2].set_ylabel('AUC', fontsize=18)
    axes[2].legend()

    plt.tight_layout()
    plt.show()

    return Epochs
```

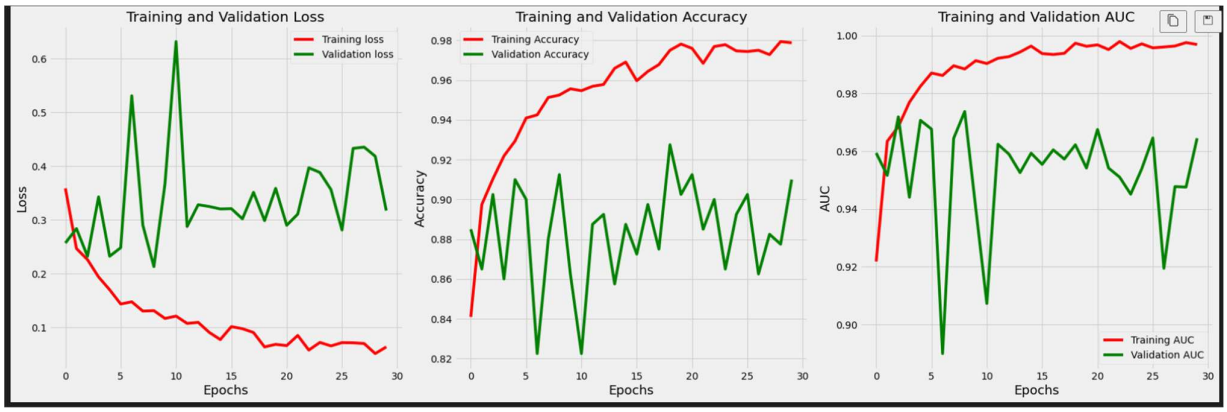
```python
def predictor(test_ds):
    y_pred = []
    y_true = test_ds.labels
    classes = list(test_ds.class_indices.keys())
    class_count = len(classes)
    errors = 0
    preds = tf.argmax(net.predict(test_ds), axis=1)
    tests = len(preds)
    for i in range(tests):
        pred_index = preds[i]
        true_index = test_ds.labels[i]
        if pred_index != true_index:
            errors += 1
        y_pred.append(pred_index)

    acc = (1 - errors / tests) * 100
    msg = f'there were {errors} errors in {tests} tests for an accuracy of {acc:6.2f}%'
    print(msg)
    ypred = np.array(y_pred)
    ytrue = np.array(y_true)
    f1score = f1_score(ytrue, ypred, average='weighted') * 100


    cm = confusion_matrix(y_true, y_pred)

    plt.figure(figsize=(16, 8))
    plt.subplot(1, 2, 1)
    sns.heatmap(cm, annot=True, vmin=0, fmt='g', cmap='Blues', cbar=False)
    plt.xticks(np.arange(class_count) + .5, classes, rotation=90, fontsize=14)
    plt.yticks(np.arange(class_count) + .5, classes, rotation=0, fontsize=14)
    plt.xlabel("Predicted", fontsize=14)
    plt.ylabel("Actual", fontsize=14)
    plt.title("Confusion Matrix")

    plt.subplot(1, 2, 2)
    sns.set(font_scale=1.6)
    sns.heatmap(cm / np.sum(cm), annot=True, fmt='.1%')
    plt.xlabel('Predicted Label', fontsize=14)
    plt.xticks(fontsize=14)
    plt.ylabel('True Label', fontsize=14)
    plt.yticks(fontsize=14)

    plt.show()

    clr = classification_report(y_true, y_pred, target_names=classes, digits=4)
    print("Classification Report:\n----------------------\n", clr)

    return f1score

f1score = predictor(test_ds)
```

```python
# -------- GAN MODEL --------
discriminator.trainable = False  # Freeze for GAN training

input_gray = Input(shape=(32, 32, 1))
generated_color = generator(input_gray)
decision = discriminator(generated_color)

gan = Model(input_gray, decision)
gan.compile(loss="binary_crossentropy", optimizer="adam")
```

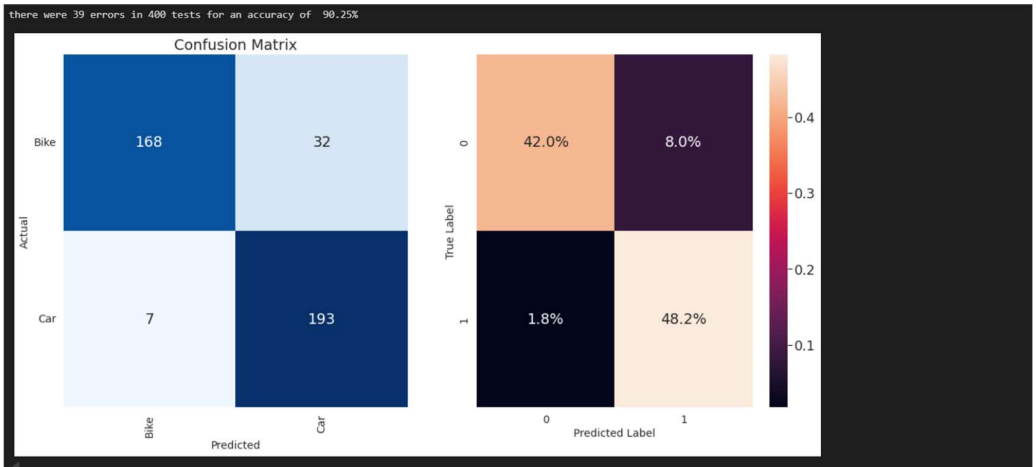
Output:

```
Found 3200 validated image filenames belonging to 2 classes.          for train generator
Found 400 validated image filenames belonging to 2 classes.           for valid generator
Found 400 validated image filenames belonging to 2 classes.           for test generator
test batch size:  80 test steps:  5 number of classes :  2
```

```
200/200 [==============================] - 28s 89ms/step - loss: 0.3587 - accuracy: 0.8409 - auc: 0.9218 - val_loss: 0.2568 - val_accuracy: 0.8850 - val_auc: 0.9594
Epoch 2/30
200/200 [==============================] - 17s 86ms/step - loss: 0.2465 - accuracy: 0.8975 - auc: 0.9634 - val_loss: 0.2837 - val_accuracy: 0.8650 - val_auc: 0.9516
Epoch 3/30
200/200 [==============================] - 16s 81ms/step - loss: 0.2258 - accuracy: 0.9103 - auc: 0.9685 - val_loss: 0.2321 - val_accuracy: 0.9025 - val_auc: 0.9719
Epoch 4/30
200/200 [==============================] - 17s 83ms/step - loss: 0.1936 - accuracy: 0.9219 - auc: 0.9769 - val_loss: 0.3428 - val_accuracy: 0.8600 - val_auc: 0.9440
Epoch 5/30
200/200 [==============================] - 16s 79ms/step - loss: 0.1698 - accuracy: 0.9294 - auc: 0.9825 - val_loss: 0.2322 - val_accuracy: 0.9100 - val_auc: 0.9707
Epoch 6/30
200/200 [==============================] - 17s 86ms/step - loss: 0.1432 - accuracy: 0.9409 - auc: 0.9871 - val_loss: 0.2481 - val_accuracy: 0.9000 - val_auc: 0.9676
Epoch 7/30
200/200 [==============================] - 16s 79ms/step - loss: 0.1473 - accuracy: 0.9425 - auc: 0.9862 - val_loss: 0.5312 - val_accuracy: 0.8225 - val_auc: 0.8899
Epoch 8/30
200/200 [==============================] - 17s 83ms/step - loss: 0.1300 - accuracy: 0.9513 - auc: 0.9896 - val_loss: 0.2901 - val_accuracy: 0.8800 - val_auc: 0.9644
Epoch 9/30
200/200 [==============================] - 16s 79ms/step - loss: 0.1309 - accuracy: 0.9525 - auc: 0.9885 - val_loss: 0.2128 - val_accuracy: 0.9125 - val_auc: 0.9737
Epoch 10/30
200/200 [==============================] - 17s 85ms/step - loss: 0.1163 - accuracy: 0.9556 - auc: 0.9914 - val_loss: 0.3670 - val_accuracy: 0.8625 - val_auc: 0.9401
Epoch 11/30
200/200 [==============================] - 16s 81ms/step - loss: 0.1207 - accuracy: 0.9547 - auc: 0.9904 - val_loss: 0.6321 - val_accuracy: 0.8225 - val_auc: 0.9072
Epoch 12/30
200/200 [==============================] - 16s 81ms/step - loss: 0.1070 - accuracy: 0.9569 - auc: 0.9922 - val_loss: 0.2875 - val_accuracy: 0.8875 - val_auc: 0.9624
Epoch 13/30
200/200 [==============================] - 16s 81ms/step - loss: 0.1090 - accuracy: 0.9578 - auc: 0.9927 - val_loss: 0.3282 - val_accuracy: 0.8925 - val_auc: 0.9590
...
Epoch 29/30
200/200 [==============================] - 17s 80ms/step - loss: 0.0509 - accuracy: 0.9794 - auc: 0.9976 - val_loss: 0.4185 - val_accuracy: 0.8775 - val_auc: 0.9476
Epoch 30/30
200/200 [==============================] - 17s 83ms/step - loss: 0.0633 - accuracy: 0.9787 - auc: 0.9969 - val_loss: 0.3172 - val_accuracy: 0.9100 - val_auc: 0.9645
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```




```
there were 39 errors in 400 tests for an accuracy of  90.25%
```




```
Classification Report:
----------------------
              precision    recall  f1-score   support

        Bike     0.9600    0.8400    0.8960       200
         Car     0.8578    0.9650    0.9082       200

    accuracy                         0.9025       400
   macro avg     0.9089    0.9025    0.9021       400
weighted avg     0.9089    0.9025    0.9021       400
```